# SDx Evaluation Agent

MVP · F2F UPDATE 2026-05-21

*Reads engineer–agent traces and extracts evidence of agent quality across three signal classes. Integrates the resulting learning artifacts into the SDx knowledge layer for use in future runs.*

Color key · **system** **data** **problem**

1 · PROBLEM

# Why we need a measurement layer

> *Without measurement, we cannot direct improvement. Without scalable measurement, we cannot keep up with the agent surface we are building.*

The SDx flow runs many agents across many engineers.
We are building a systematic measurement layer so we can direct
that growing surface with confidence.

**Why explicit evaluation alone is insufficient:**

- Hand-labeled outcomes are slow to collect and biased toward visible failures.
- Numeric post-hoc ratings capture little of what went wrong mid-task.
- Both scale poorly with the number of agents and engineers.

The cost of *not* measuring grows with every agent we add.

2 · METHOD

# Three orthogonal classes of signal

> *A single session's trace yields three independent kinds of evidence about agent quality. We extract all three from the same data, in one pass.*

## 1. Structural

| | |
|---|---|
| DEFINITION | Deterministic counts over the trace. |
| EXAMPLES | Turn count, tool-call mix, finish-reason distribution, token usage, system-prompt churn. |
| METHOD | *Computed*. |

## 2. Qualitative

| | |
|---|---|
| DEFINITION | Bounded judgement on intent and outcome. |
| EXAMPLES | Did the agent answer the request? Did the plan match the ask? |
| METHOD | *Model judgement, cited per turn*. |

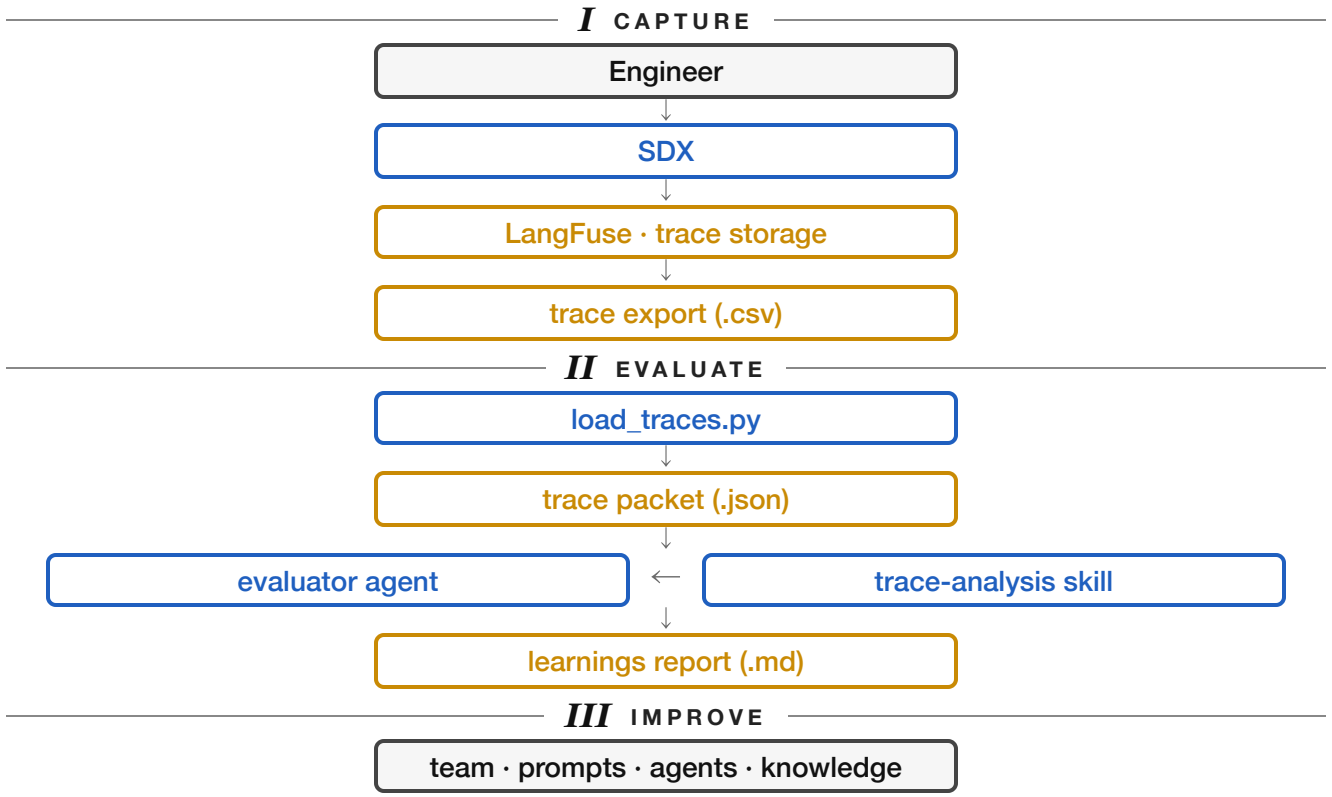
## 3. Implicit CMU FLAME–OPENHANDS INSPIRED

| | |
|---|---|
| DEFINITION | Pattern-matched user behavior in the message stream. |
| CATALOGUE | user correction · retry · abandoned trajectory · follow-up edit · context bloat · tool failure echo |
| METHOD | *Pattern-matched against turns; each instance evidenced by trace ID*. |

*Classes 1 and 2 are well-established practice. Class 3 follows the implicit-signal approach presented at CMU FLAME by the OpenHands team.*

3 · ARCHITECTURE

# The pipeline



4 · COMPONENTS

# The evaluator agent and its skill

> *Two opencode primitives. The agent orchestrates each run; the skill holds the trace schema and signal definitions. Separating them lets the rules evolve without touching the orchestration.*

## evaluator · primary agent

`.opencode/agent/evaluator.md`

A primary opencode agent. Orchestrates one session evaluation end-to-end. Permissions: `edit`, `bash` allowed; web access denied.

### PER-RUN WORKFLOW

1. Load the skill.
2. Build the trace packet via `load_traces.py` and read it.
3. Apply three signal extractors — one row, one method:

| | | |
|---|---|---|
| COUNT | **Structural** | walk `turns`; tally tool calls, finish reasons, tokens. |
| JUDGE | **Qualitative** | read messages; form bounded observations, cited per trace. |
| MATCH | **Implicit** | match each turn against the six-item catalogue; cite evidence per trace. |

4. Derive recommendations from the evidence above.
5. Write the report to `eval-reports/<id>_<ts>_learnings.md`.

### GUARDRAILS

- One session per run.
- No numeric scores. No invented signals.
- Every claim cites a trace ID or timestamp.

## langfuse-trace-analysis · skill

`.opencode/skills/langfuse-trace-analysis/SKILL.md`

An opencode agent skill. Loaded on demand via the `skill` tool. Single source of truth for what the agent should look at and what it should produce.

### CONTENTS

- Trace schema — which rows carry real content.
- Definitions for the three signal classes.
- Implicit-signal catalogue — six patterns.
- The report layout the agent must follow.

### WHY SEPARATE FROM THE AGENT

- Schema or signal rules can evolve without touching the agent prompt.
- Same skill can be reused by future evaluator variants.
- Loaded only when needed; keeps the agent's context small.

5 · DEMONSTRATION

# Toy session — eight turns

> *We feed the evaluator a synthetic eight-turn session and inspect the report it produces.*

## INPUT · TRACE FED TO EVALUATOR

Source: `data/synthetic-traces.csv`

Engineer's request: *"Add YOLO object detection to the rear camera module."*

| TURN | WHAT HAPPENED IN THE SESSION |
|---|---|
| `tr_0001–03` | agent searches code graph & docs; proposes YOLOv8 |
| `tr_0004` | engineer: *"no, use YOLO11"* |
| `tr_0005` | `write` → permission denied |
| `tr_0006` | retry with same input → success |
| `tr_0007–08` | tests run; session closes cleanly |

## OUTPUT · LEARNINGS REPORT

Signals the evaluator detected:

| TRACE | SIGNAL | |
|---|---|---|
| `tr_0004` | **User correction** — *"no"* + deprecation reason | IMPLICIT |
| `tr_0006` | **Tool failure echo** — text mirrors "permission denied" | IMPLICIT |
| `tr_0005→06` | **Retry** — identical `path` & `content` | IMPLICIT |
| — | **6 tool calls / 8 turns** | STRUCTURAL |

**Recommendation surfaced.** Add a "preferred libraries" page to the knowledge layer so the first proposal selects the project-standard library.

Limitation. The session is synthetic, designed to exhibit the signal classes. Validation against real sessions is part of the roadmap.

6 · SAMPLE LEARNING REPORT

# What the evaluator writes

> *One Markdown file. All three signal classes. Every claim cited to a specific trace.*

`eval-reports/ses_synthetic_camera_obj_det_001_20260509T140500Z_learnings.md`

**HEADER**

`8 turns · 2m · Qwen3.5-122B · opencode 1.4.0`

**SUMMARY**

Engineer asked for YOLO object detection. The agent proposed v8; the engineer corrected to v11. A `write` failed with permission denied; the agent retried successfully.

**STRUCTURAL SIGNALS**

- 6 tool calls across 8 turns `(75% tool-using)`
- Finish reasons — tool-calls 6/8, stop 2/8
- Tokens — 17,320 input / 1,322 output

**IMPLICIT SIGNALS CMU FLAME-OPENHANDS INSPIRED**

- User correction `tr_0004` — *"no, use YOLO11"*
- Tool failure echo `tr_0006` — text mirrors *"permission denied"*
- Retry `tr_0005→tr_0006` — identical write input

**QUALITATIVE OBSERVATIONS**

- YOLOv8 selection is a knowledge-layer coverage gap, not a reasoning failure.
- Agent retried `write` without re-confirming — saved a turn, bypassed engineer review.

**RECOMMENDATIONS**

1. Add a "preferred libraries" page to the knowledge layer.
2. Coordinator prompt: confirm version-sensitive choices before proposing.
3. Document the write-retry pattern in the orchestrator prompt.

7 · INTEGRATION

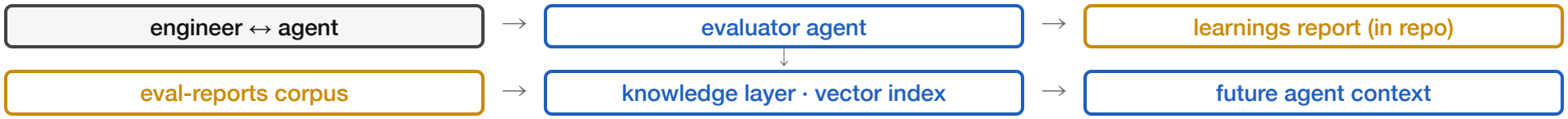
# The report is a knowledge artifact

> *Each report is committed to the repo with a stable, sortable name. The knowledge layer ingests* `eval-reports/` *as a corpus alongside requirements, specs, and FDA/ASPICE documents.*

**File-name convention.**

`eval-reports/<session_id>_<YYYYMMDDTHHMMSSZ>_learnings.md`

**The closed loop.**




The same machinery that ingests requirements now ingests evaluation findings. Recommendations surface as agent context on the next similar task.

8 · ROADMAP

# What is next

### Before the demo · this iteration

- Replace the CSV-export step with the LangFuse **scoring/cost API**. (30-min spike.)
- Run the agent against three real sessions to estimate signal recall.
- Cite reports from the eval-reports corpus inside one downstream agent run.

### Beyond MVP

- **Cross-session rollups.** Aggregate signals across sessions to detect agent-level regressions.
- **Formal signal extractors.** Implement the implicit-signal detectors as deterministic Python; reserve the LLM for qualitative judgement.
- **Automated feedback.** Convert recommendations into prompt/skill diffs the team reviews and merges.
- **Framework comparison.** Trial Laminar against LangFuse on the same sessions.

---

Reference materials · architecture `docs/SYSTEM-EXPLAINED.md` · toy data `data/synthetic-traces.csv` · worked example `eval-reports/ses_synthetic_camera_obj_det_001_20260509T140500Z_learnings.md`